

课程笔记

Planning Agent: Plan-Execute-RePlan

基于公开课程资料整理

2025

Plan Agent

plan-execute-replan

todo list

视频作者/频道: 五道口纳什

发布日期: 2025

视频时长: 约 14 分钟

视频链接: <https://www.bilibili.com/video/BV1qa43z5EBJ>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1	Plan-Execute-RePlan 概述	2
1.1	核心理念	2
1.2	为什么 Plan-Execute-RePlan 有效	2
1.3	本章小结	2
2	Agentic Workflow vs. Autonomous Agent	3
2.1	概念区分	3
2.2	OpenAI 关于 Reasoning Model 的最佳实践	3
2.3	本章小结	3
3	LangGraph 实现详解	3
3.1	状态定义	3
3.2	Plan 节点	3
3.3	Execute 节点	4
3.4	RePlan 节点	4
3.5	Graph 结构	4
3.6	本章小结	4
4	经济性与模型组合策略	4
4.1	本章小结	5
5	总结与延伸	5

1 Plan-Execute-RePlan 概述

1.1 核心理念

本期介绍了一个非常 general 且在真实科研和工作场景中广泛使用的 Agentic Workflow——**Dynamic Plan-and-Solve**，即 Plan-Execute-RePlan 三步循环。

Plan-Execute-RePlan 三步曲

1. **Plan**: 拿到用户的复杂 query/task，先制定一个自顶向下的 sub-goal 分解，输出一个 List of Steps (To-Do List)
2. **Execute**: 逐步消化 Plan 中的每一个 step，通过工具调用获取 outcome 和 feedback
3. **RePlan**: 基于执行结果和 feedback，动态更新剩余的 steps

这种模式已被广泛采用——Cursor、Claude Code、Codex 等 Coding Agent 都支持 Plan Mode，先列 To-Do List，再边执行边更新。

1.2 为什么 Plan-Execute-RePlan 有效

结构化 Workflow 有助于：

- 结构化模型的思考过程
- 降低对复杂问题求解的整体复杂度
- 自顶向下分解 + 自底向上反馈形成闭环

Top-Down Plan vs. Bottom-Up Execute

- **Plan (Top-Down)**: 自顶向下的 sub-goal 分解，不涉及和环境的真实交互，是” 刚中之脑”——想象出来的 sub-goal 序列
- **Execute (Bottom-Up)**: 和环境真实发生交互，拿到真实的 feedback
- **RePlan**: 基于真实交互的结果，动态修正计划

1.3 本章小结

Plan-Execute-RePlan 是一个非常 general 且经过实践验证的 Agentic Workflow，其核心价值在于将复杂问题结构化分解，并通过动态反馈机制弥合规划与执行之间的 gap。

2 Agentic Workflow vs. Autonomous Agent

2.1 概念区分

Anthropic 对 Agent 的定义

根据 Anthropic 年初关于 Agent 的定义，Plan-Execute-RePlan 属于**预定义的 Agentic Workflow**，而非 Autonomous Agent。但这并不意味着 Agentic Workflow 一定比 Autonomous Agent 差——Agentic 是一个光谱，Autonomous Agent 是终极形态。

预定义 Workflow 的优势在于**可控、可靠、可预测**。完全自主的 Agent（纯 ReAct 范式）受限于模型能力，可能表现不佳。

2.2 OpenAI 关于 Reasoning Model 的最佳实践

OpenAI API 官方文档中关于 Reasoning Model 的定位：

- **O 系列推理模型**：适合做 Planner——可以对复杂任务执行更长时间、更深入的思考，更有效地制定策略
- **GPT 非推理模型**：适合做 Executor——追求低延迟、低成本效益，直接执行具体任务

2.3 本章小结

Agentic Workflow 胜在可控可靠，是当前实际应用中的主流选择。强模型做规划、弱模型做执行的策略与 OpenAI 的官方建议一致。

3 LangGraph 实现详解

3.1 状态定义

全局共享的状态（State）包含四个字段：

- **input**：用户的问题/任务
- **plan**：形成的 Plan List（sub-goal 序列）
- **past_steps**：已经执行过的步骤及其结果（追加式）
- **response**：最终给用户的回答

3.2 Plan 节点

Plan 的 System Prompt 设计非常 general（与具体任务无关）：

For the given objective, come up with a simple step by step plan. This plan should involve individual tasks, that if executed correctly will yield the correct answer. Do not add any superfluous steps. The result of the final step should be the final answer. Make sure that each step has all the information needed – do not skip steps.

输出是结构化的 Steps 列表。

3.3 Execute 节点

Execute 使用 ReAct Agent (`create_react_agent`) 来消化 Plan 中的每一个 step。一个 step 可能对应多个工具调用。执行结果追加到 `past_steps` 中。

3.4 RePlan 节点

RePlan 的输入包含三部分：

1. 用户的原始目标 (objective)
2. 原始的 Plan
3. 已经执行过的 steps 及其结果

RePlan Prompt 的关键约束

- 如果没有更多步骤需要执行，直接 return to user
- 否则，只添加仍需执行的步骤到 Plan 中
- 不要返回已经执行过的 steps 作为新 Plan

3.5 Graph 结构

三个节点 + 条件边：

1. **Planner** → **Agent** (ReAct Execute) → **RePlan**
2. RePlan 有一条条件边：决定是结束 (response) 还是继续执行 (回到 Agent)

3.6 本章小结

LangGraph 实现展示了如何将 Plan-Execute-RePlan 落地为一个清晰的三节点 Graph，其中 Plan 和 RePlan 决定了整体质量上限，Execute 负责具体的工具调用。

4 经济性与模型组合策略

强弱模型组合原则

- **Planning 和 RePlan**：使用强模型（如推理模型），因为规划决定了整个 Workflow 的上限
- **Execute**：使用弱模型（如 GPT-4.1 Nano/Mini），因为已经告诉模型该怎么做，只需一步步执行

这与 OpenAI 对推理模型和非推理模型的定位完全一致。

如果 Planning 环节特别重要，还可以引入 **Generator-Critique** 模式来形成更稳固可靠的 Plan——不要预期模型一次性就能提出一个很好的 Plan。

4.1 本章小结

合理的模型组合策略可以在成本和效果之间取得平衡：贵的模型做规划，便宜的模型做执行。

5 总结与延伸

1. **Plan-Execute-RePlan** 是一个非常 **general** 且有效的 **Workflow**，适用于各种复杂任务场景，已被广泛验证。
2. **自顶向下分解 + 自底向上反馈**：Plan 是”刚中之脑”的想象，Execute 是真实交互，RePlan 弥合两者之间的 gap。
3. **经济性考量**：强模型做规划、弱模型做执行，与 OpenAI 的官方建议一致。
4. **Plan 的质量决定上限**：如果任务复杂度高，可以在 Planning 阶段使用更强的模型或引入 Generator-Critique 机制。
5. **从 Agentic Workflow 到 Autonomous Agent**：目前的实践证明，预定义的 Workflow 在可控性和可靠性上优于完全自主的 Agent。